FULL CODE # In Case

```
# ==========================================
# DRAGON'S LAIR Battle
# ==========================================

class Character
  property name      : String
  property hp        : Int32
  property max_hp    : Int32
  property mp        : Int32
  property max_mp    : Int32
  property attack    : Int32
  property defense   : Int32
  property level     : Int32
  property exp       : Int32
  property potions   : Int32
  property gold      : Int32
  property status    : Symbol  # :normal, :poisoned, :burning,
:stunned

  def initialize(@name : String, @max_hp : Int32, @max_mp :
Int32,
                 @attack : Int32 = 10, @defense : Int32 = 5)
    @hp      = @max_hp
    @mp      = @max_mp
    @level   = 1
    @exp     = 0
```

```
    @potions = 3
    @gold    = 0
    @status  = :normal
  end

  def is_alive?;  @hp > 0;          end
  def has_mp?(n); @mp >= n;         end

  def take_damage(amount : Int32)
    reduced = [amount - @defense, 1].max   # Defense reduces
damage (min 1)
    @hp = [@hp - reduced, 0].max
    reduced
  end

  def raw_damage(amount : Int32)           # Bypasses defense
(poison, fire)
    @hp = [@hp - amount, 0].max
    amount
  end

  def heal(amount : Int32)
    old = @hp
    @hp = [@hp + amount, @max_hp].min
    @hp - old                              # Return actual
healed
  end
```

```
def consume_mp(amount : Int32)
  @mp = [@mp - amount, 0].max
end

def apply_status_damage                    # Called each turn
  case @status
  when :poisoned
    dmg = raw_damage(5)
    puts "    ☠  #{@name} writes in poison... (-#{dmg} HP)"
  when :burning
    dmg = raw_damage(8)
    puts "    🔥 #{@name} is burning... (-#{dmg} HP)"
  end
end

def status_label
  case @status
  when :poisoned then " [POISONED]"
  when :burning  then " [BURNING]"
  when :stunned  then " [STUNNED]"
  else ""
  end
end

def hp_bar(width = 20) : String
  filled = [((@hp.to_f / @max_hp) * width).round.to_i, 0].max
  empty  = [width - filled, 0].max
```

```
    "▓" * filled + "░" * empty
  end
end

class InventoryItem
  property name    : String
  property count   : Int32
  property effect  : String

  def initialize(@name : String, @count : Int32, @effect :
String); end
end

# ──────────────────────────────────────────
# DISPLAY HELPERS
# ──────────────────────────────────────────
def clear_screen
  system "cmd /c cls"
end

def print_status(player : Character, boss : Character)
  puts ""
  puts "=" * 50
  puts "  Lv#{player.level}
#{player.name}#{player.status_label}"
  puts "  HP [#{player.hp_bar}] #{player.hp}/#{player.max_hp}"
  puts "  MP [#{"◆" * (player.mp // 5)}#{"◇" * ((player.max_mp -
player.mp) // 5)}] #{player.mp}/#{player.max_mp}"
```

```
  puts "  Gold: #{player.gold}  |  Potions: #{player.potions}"
  puts "-" * 50
  puts "  #{boss.name}#{boss.status_label}"
  puts "  HP [#{boss.hp_bar}] #{boss.hp}/#{boss.max_hp}"
  puts "=" * 50
  puts ""
end

def dramatic_pause(msg : String, delay : Float64 = 0.05)
  msg.each_char { |c| print c; STDOUT.flush; sleep delay.seconds }
  puts ""
end

# ──────────────────────────────────────────
# ABILITY DEFINITIONS
# ──────────────────────────────────────────
struct Ability
  getter name    : String
  getter mp_cost : Int32
  getter desc    : String

  def initialize(@name : String, @mp_cost : Int32, @desc : String); end
end

ABILITIES = [
```

```
  Ability.new("Slash",          0,  "A quick sword slash  (10-20 dmg)"),
  Ability.new("Magic Missile",  10, "Arcane bolt          (25-40 dmg)"),
  Ability.new("Thunder Strike", 20, "Lightning attack      (35-55 dmg)"),
  Ability.new("Drain Life",     15, "Steal HP from enemy   (20-30 dmg, heal half)"),
  Ability.new("Drink Potion",   0,  "Restore 40-60 HP"),
  Ability.new("Taunt",          5,  "Lower dragon defense for 2 turns"),
]

# ──────────────────────────────────────────────
# GAME SETUP
# ──────────────────────────────────────────────
player = Character.new("Crystal Knight", 120, 40, attack: 12, defense: 6)
dragon = Character.new("Inferno Dragon",  200, 0,  attack: 18, defense: 4)

turn       = 0
taunted    = 0   # How many turns dragon defense is reduced
combo      = 0   # Consecutive hits without taking damage
high_score = 0

clear_screen
puts ""
dramatic_pause("  ⚔  You have entered the DRAGON'S LAIR!  ⚔", 0.04)
```

```
puts ""
puts "  The #{dragon.name} awakens, eyes blazing..."
puts "  Survive and claim the Crystal Throne!"
sleep 2.5.seconds

# ──────────────────────────────────────────────
# BATTLE LOOP
# ──────────────────────────────────────────────
while player.is_alive? && dragon.is_alive?
  turn += 1
  clear_screen
  print_status(player, dragon)

  # Status effects tick at the start of each turn
  player.apply_status_damage unless player.status == :normal
  break unless player.is_alive?

  # Display taunted reminder
  puts "  * Dragon is weakened! (#{taunted} turns remain)" if
taunted > 0

  # ── PLAYER TURN ──────────────────────────
  if player.status == :stunned
    puts "  ⚡ You are stunned and skip your turn!"
    player.status = :normal
    sleep 1.5.seconds
  else
```

```
    puts "Choose your action:"
    puts "  [1] ⚔  #{ABILITIES[0].name.ljust(18)} | #{ABILITIES[0].desc}"
    puts "  [2] ✨ #{ABILITIES[1].name.ljust(18)} | #{ABILITIES[1].desc} (MP: #{player.mp}/#{player.max_mp})"
    puts "  [3] ⚡ #{ABILITIES[2].name.ljust(18)} | #{ABILITIES[2].desc} (MP: #{player.mp}/#{player.max_mp})"
    puts "  [4] 💙 #{ABILITIES[3].name.ljust(18)} | #{ABILITIES[3].desc} (MP: #{player.mp}/#{player.max_mp})"
    puts "  [5] 🩹 #{ABILITIES[4].name.ljust(18)} | Restore HP (Potions: #{player.potions})"
    puts "  [6] 😭 #{ABILITIES[5].name.ljust(18)} | #{ABILITIES[5].desc}"
    print "\nAction (1-6): "

    action = (gets || "").strip
    puts "\n" + "-" * 50

    case action
    when "1"
      damage = rand(10..20) + (combo > 2 ? 5 : 0)  # Combo bonus
      effective = dragon.take_damage(damage)
      combo += 1
      dramatic_pause("  ⚔  Slash! You deal #{effective} damage! (#{combo > 1 ? "Combo x#{combo}!" : ""})")
    when "2"
      if player.has_mp?(10)
        player.consume_mp(10)
        damage  = rand(25..40)
```

```
        effective = dragon.take_damage(damage)
        combo += 1
        dramatic_pause("  ✨ Magic Missile! #{effective} arcane
damage!")
      else
        puts "  Not enough MP!"
        combo = 0
      end
    when "3"
      if player.has_mp?(20)
        player.consume_mp(20)
        damage = rand(35..55)
        effective = dragon.take_damage(damage)
        combo += 1
        dramatic_pause("  ⚡ THUNDER STRIKE! #{effective}
lightning damage!")
        if rand(1..4) == 1
          dragon.status = :stunned
          puts "  ⚡ The dragon is STUNNED!"
        end
      else
        puts "  Not enough MP!"
        combo = 0
      end
    when "4"
      if player.has_mp?(15)
        player.consume_mp(15)
```

```
        damage = rand(20..30)
        effective = dragon.take_damage(damage)
        # FIX: Using integer division (//)
        stolen = player.heal(effective // 2)
        combo += 1
        dramatic_pause("  💙 Drain Life! #{effective} dmg,
recovered #{stolen} HP!")
      else
        puts "  Not enough MP!"
        combo = 0
      end
    when "5"
      if player.potions > 0
        player.potions -= 1
        healed = player.heal(rand(40..60))
        player.status = :normal
        combo = 0
        dramatic_pause("  🧪 Potion! Recovered #{healed} HP!
Cleansed status.")
      else
        puts "  No potions left!"
      end
    when "6"
      if player.has_mp?(5)
        player.consume_mp(5)
        taunted = 3
        combo = 0
```

```
        dramatic_pause("  😤 You taunt the dragon — its guard
drops for 3 turns!")
      else
        puts "  Not enough MP to taunt!"
      end
    else
      puts "  Invalid choice! You hesitated!"
      combo = 0
    end

    sleep 1.5.seconds
  end

  break unless dragon.is_alive?
  high_score = [high_score, combo].max

  # ── DRAGON TURN ──────────────────────────
  if dragon.status == :stunned
    puts "\n  ⚡ The Dragon is stunned and cannot attack!"
    dragon.status = :normal
    sleep 1.5.seconds
    taunted -= 1 if taunted > 0
    next
  end

  puts "\n  The #{dragon.name} moves..."
  sleep 0.8.seconds
```

```
  dragon_roll = rand(1..10)

  # Taunted dragon has reduced effective defense (hits harder,
but player chose the risk)
  defense_mod = taunted > 0 ? 2 : 0

  boss_damage = case dragon_roll
  when 9, 10
    dmg = rand(30..45)
    dramatic_pause("  DRAGON BREATH! Scorching flames engulf
you for #{player.take_damage(dmg + defense_mod)} damage!")
    if rand(1..3) == 1
      player.status = :burning
      puts "  You catch fire!"
    end
    dmg
  when 7, 8
    dmg = rand(15..25)
    dramatic_pause("  Tail Swipe! The dragon sweeps you for
#{player.take_damage(dmg)} damage!")
    if rand(1..4) == 1
      player.status = :poisoned
      puts "  Poisoned claw! You feel sick..."
    end
    dmg
  when 1
```

```
    dramatic_pause("  😴 The dragon roars but misses entirely!
(No damage)")
    0
  else
    dmg = rand(10..20)
    dramatic_pause("  🦷 The dragon bites you for
#{player.take_damage(dmg)} damage!")
    dmg
  end

  taunted -= 1 if taunted > 0
  combo = 0 if boss_damage > 0
  sleep 1.5.seconds
end

# ──────────────────────────────────────────────
# GAME OVER
# ──────────────────────────────────────────────
clear_screen
print_status(player, dragon)

if player.is_alive?
  player.gold += rand(100..200)
  dramatic_pause("  ★ VICTORY! You have slain the
#{dragon.name}!")
  puts "  You earned #{player.gold} gold and the Crystal Throne
is yours!"
  puts "  Turns survived: #{turn}  |  Max Combo: #{high_score}"
```

```
else
  dramatic_pause("  †  You were defeated by the #{dragon.name}...")
  puts "  You lasted #{turn} turns. Return stronger, warrior."
end

puts ""
puts "  Thanks for playing Dragon's Lair: Crystal Edition!"
```